

# LOADING THE NECESSARY LIBRARIES

In [2]:

```
import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from wordcloud import WordCloud, STOPWORDS

import nltk
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize, sent_tokenize

from bs4 import BeautifulSoup

import re, string, unicodedata

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelBinarizer

from string import punctuation

from nltk import pos_tag
from nltk.corpus import wordnet

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, Embedding, LSTM, Dropout
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing import text, sequence
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

# IMPORTING THE ISOT DATASET

```
In [3]: real = pd.read_csv("/Users/moe/Desktop/ISOT_News_Dataset/True.csv")
fake = pd.read_csv("/Users/moe/Desktop/ISOT_News_Dataset/Fake.csv")
```

## DATA VISUALIZATION AND PREPROCESSING

```
In [4]: real.head()
```

```
Out[4]:
```

|   | title                                             | text                                              | subject      | date              |
|---|---------------------------------------------------|---------------------------------------------------|--------------|-------------------|
| 0 | As U.S. budget fight looms, Republicans flip t... | WASHINGTON (Reuters) - The head of a conservat... | politicsNews | December 31, 2017 |
| 1 | U.S. military to accept transgender recruits o... | WASHINGTON (Reuters) - Transgender people will... | politicsNews | December 29, 2017 |
| 2 | Senior U.S. Republican senator: 'Let Mr. Muell... | WASHINGTON (Reuters) - The special counsel inv... | politicsNews | December 31, 2017 |
| 3 | FBI Russia probe helped by Australian diplomat... | WASHINGTON (Reuters) - Trump campaign adviser ... | politicsNews | December 30, 2017 |
| 4 | Trump wants Postal Service to charge 'much mor... | SEATTLE/WASHINGTON (Reuters) - President Donal... | politicsNews | December 29, 2017 |

```
In [5]: fake.head()
```

```
Out[5]:
```

|   | title                                            | text                                              | subject | date              |
|---|--------------------------------------------------|---------------------------------------------------|---------|-------------------|
| 0 | Donald Trump Sends Out Embarrassing New Year'... | Donald Trump just couldn t wish all Americans ... | News    | December 31, 2017 |
| 1 | Drunk Bragging Trump Staffer Started Russian ... | House Intelligence Committee Chairman Devin Nu... | News    | December 31, 2017 |
| 2 | Sheriff David Clarke Becomes An Internet Joke... | On Friday, it was revealed that former Milwauk... | News    | December 30, 2017 |
| 3 | Trump Is So Obsessed He Even Has Obama's Name... | On Christmas day, Donald Trump announced that ... | News    | December 29, 2017 |
| 4 | Pope Francis Just Called Out Donald Trump Dur... | Pope Francis used his annual Christmas Day mes... | News    | December 25, 2017 |

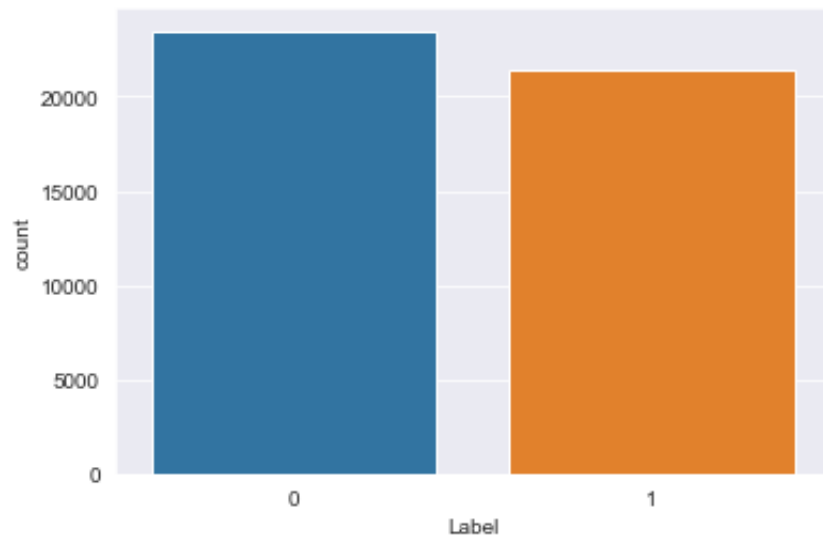
```
In [6]: real['Label'] = 1  
fake['Label'] = 0
```

```
In [7]: df = pd.concat([real, fake])
```

```
In [8]: sns.set_style("darkgrid")  
sns.countplot(df.Label)
```

/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/seaborn/\_decorators.py:36: FutureWarning  
: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.  
warnings.warn(

```
Out[8]: <AxesSubplot:xlabel='Label', ylabel='count'>
```



```
In [9]: df.head()
```

| Out[ 9 ] : |                                                   | title                                             | text         | subject           | date | Label |
|------------|---------------------------------------------------|---------------------------------------------------|--------------|-------------------|------|-------|
| 0          | As U.S. budget fight looms, Republicans flip t... | WASHINGTON (Reuters) - The head of a conservat... | politicsNews | December 31, 2017 | 1    |       |
| 1          | U.S. military to accept transgender recruits o... | WASHINGTON (Reuters) - Transgender people will... | politicsNews | December 29, 2017 | 1    |       |
| 2          | Senior U.S. Republican senator: 'Let Mr. Muell... | WASHINGTON (Reuters) - The special counsel inv... | politicsNews | December 31, 2017 | 1    |       |
| 3          | FBI Russia probe helped by Australian diplomat... | WASHINGTON (Reuters) - Trump campaign adviser ... | politicsNews | December 30, 2017 | 1    |       |
| 4          | Trump wants Postal Service to charge 'much mor... | SEATTLE/WASHINGTON (Reuters) - President Donal... | politicsNews | December 29, 2017 | 1    |       |

```
In [10]: df.isna().sum()
```

```
Out[10]: title      0
text      0
subject    0
date      0
Label     0
dtype: int64
```

```
In [11]: df.title.count()
```

```
Out[11]: 44898
```

```
In [12]: df.subject.value_counts()
```

```
Out[12]: politicsNews    11272
worldnews              10145
News                   9050
politics                6841
left-news              4459
Government News        1570
US_News                 783
Middle-east            778
Name: subject, dtype: int64
```

## MERGING ALL THE TEXT DATA INTO 1 COLUMN

```
In [13]: df['text'] = df['text'] + " " + df['title']
del df['title']
del df['subject']
del df['date']
```

## CLEANING THE DATA

```
In [82]: stop = set(stopwords.words('english'))
punctuation = list(string.punctuation)
stop.update(punctuation)

def strip_html(text):
    soup = BeautifulSoup(text, "html.parser")
    return soup.get_text()

def remove_between_square_brackets(text):
    return re.sub(r'\[[^\]]*\]', '', text)

def remove_urls(text):
    return re.sub(r'http\S+', '', text)

def remove_stopwords(text):
    final_text = []
    for i in text.split():
        if i.strip().lower() not in stop:
            final_text.append(i.strip())
    return " ".join(final_text)

def clean_text(text):
    text = re.sub(r'\[.*?\]', '', text)
    text = re.sub(r'https?://\S+|www\.\S+', '', text)
    text = re.sub(r'<.*?>+', '', text)
    text = re.sub(r'%s' % re.escape(string.punctuation), '', text)
    text = re.sub(r'\n', '', text)
    text = re.sub(r'\w*\d\w*', '', text)
    text = re.sub(r'caption', '', text)
    text = re.sub(r'photo', '', text)
    text = re.sub(r'containerid', '', text)
    text = re.sub(r'function', '', text)
```

```

text = re.sub(r'modernizr', '', text)
text = re.sub(r'modernizrphone', '', text)
text = re.sub(r'modernizrmobile', '', text)
text = re.sub(r'modernizrtablet', '', text)
text = re.sub(r'typeof', '', text)
text = re.sub(r'videopinner', '', text)
text = re.sub(r'undefined', '', text)
text = re.sub(r'null', '', text)
text = str(text).lower()
return text

#Removing the noisy text
def denoise_text(text):
    text = strip_html(text)
    text = remove_between_square_brackets(text)
    text = remove_urls(text)
    text = remove_stopwords(text)
    text = clean_text(text)
    return text

#Apply function on text column
df['text'] = df['text'].apply(denoise_text)

```

## Lemmatize words

In [15]:

```

def lemmatize_words(text):
    wnl = nltk.stem.WordNetLemmatizer()
    lem = ' '.join([wnl.lemmatize(word) for word in text.split()])
    return lem

df['text'] = df['text'].apply(lemmatize_words)

```

## WORDCLOUD FOR REAL TEXT

In [16]:

```

plt.figure(figsize = (20,20)) # Text that is not Fake
wc = WordCloud(max_words = 2000 , width = 1600 , height = 800 , stopwords = STOPWORDS).generate(" ".join(df[df.Label == 'not fake']['text']))
plt.imshow(wc , interpolation = 'bilinear')

```

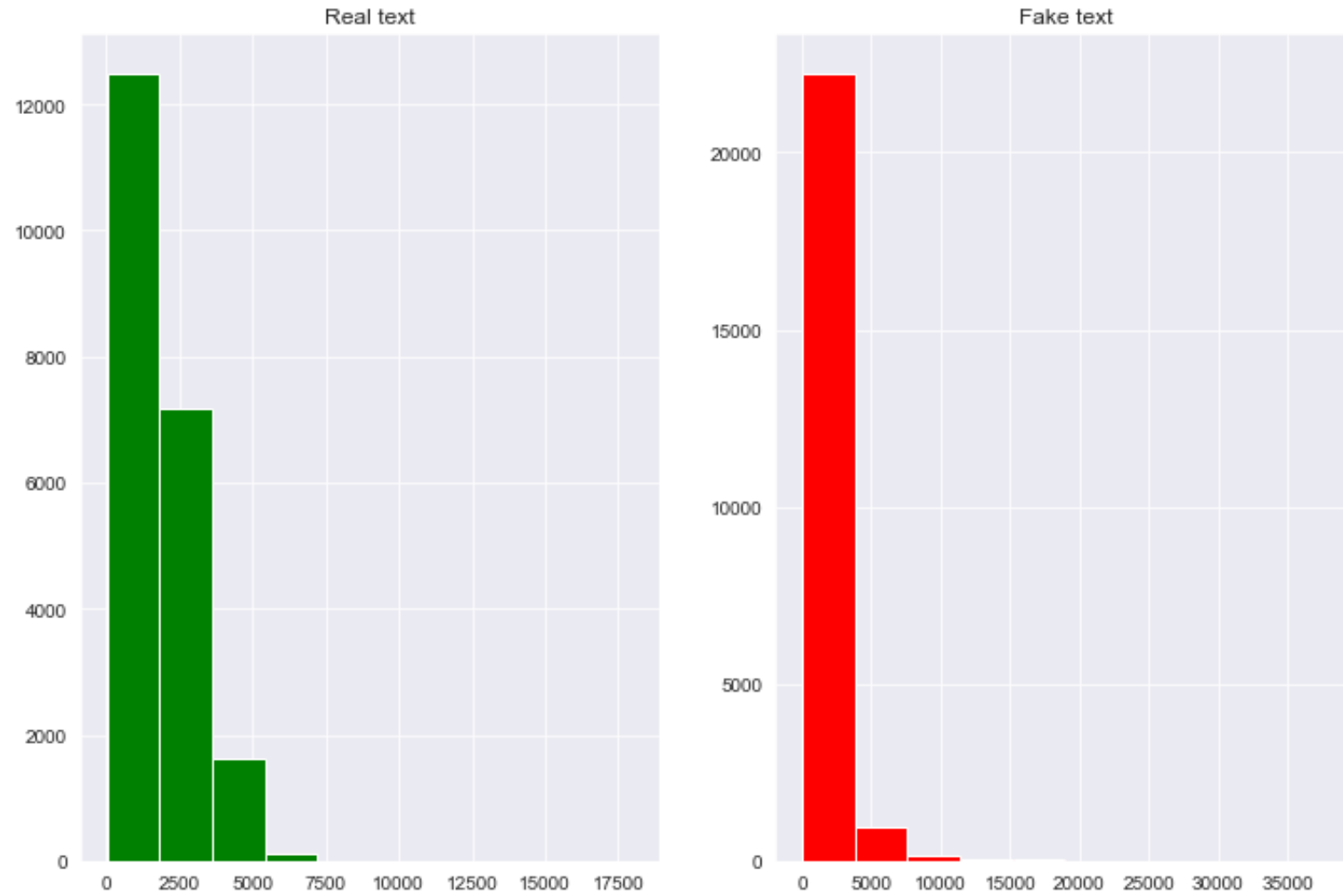




## Number of characters in texts

```
In [18]: fig,(ax1,ax2)=plt.subplots(1,2,figsize=(12,8))
text_len=df[df['Label']==1]['text'].str.len()
ax1.hist(text_len,color='green')
ax1.set_title('Real text')
text_len=df[df['Label']==0]['text'].str.len()
ax2.hist(text_len,color='red')
ax2.set_title('Fake text')
fig.suptitle('Characters in texts')
plt.show()
```

Characters in texts

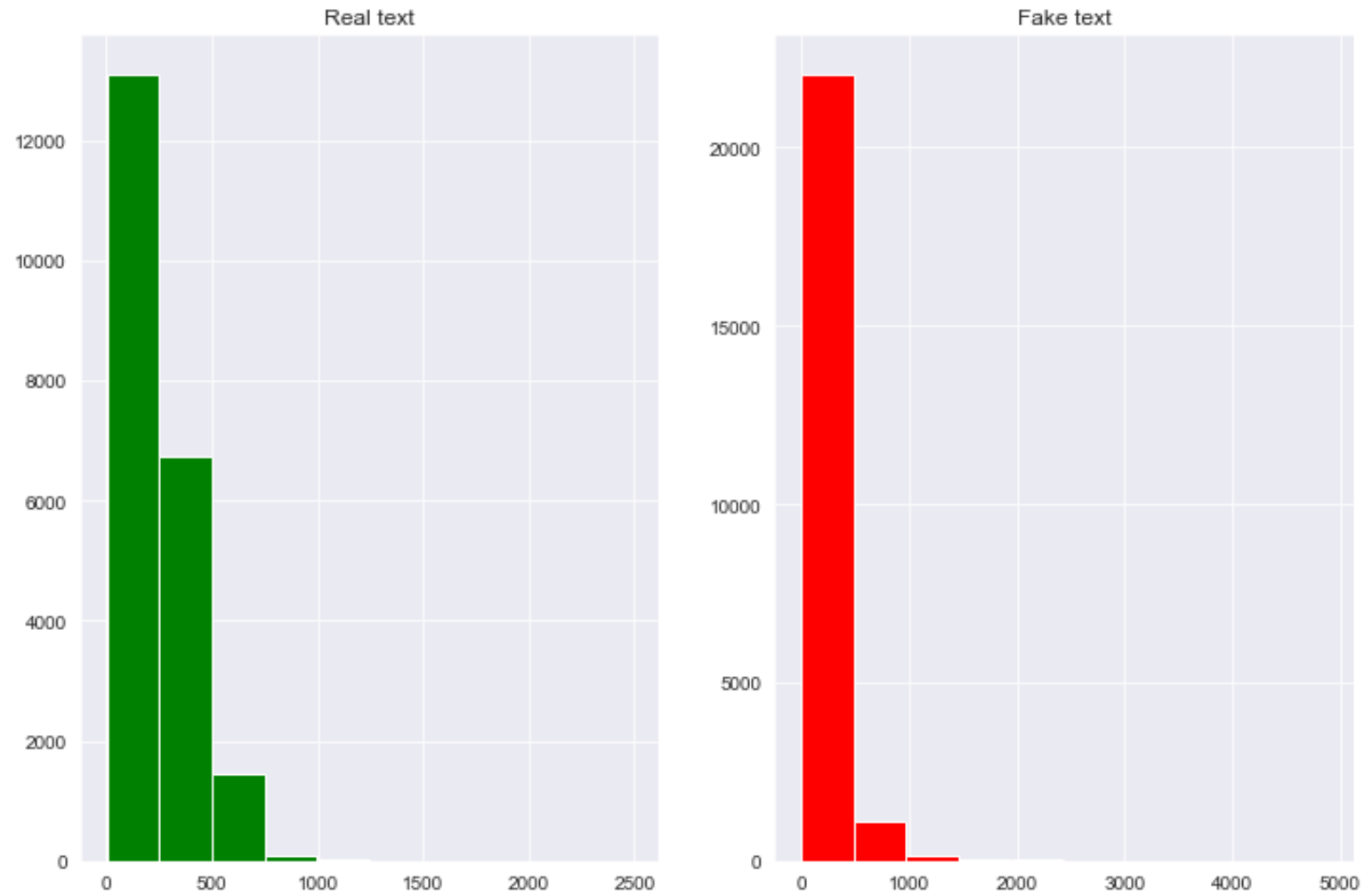


Number of words in each text

In [19]:

```
fig,(ax1,ax2)=plt.subplots(1,2,figsize=(12,8))
text_len=df[df['Label']==1]['text'].str.split().map(lambda x: len(x))
ax1.hist(text_len,color='green')
ax1.set_title('Real text')
text_len=df[df['Label']==0]['text'].str.split().map(lambda x: len(x))
ax2.hist(text_len,color='red')
ax2.set_title('Fake text')
fig.suptitle('Words in texts')
plt.show()
```

## Words in texts



Average word length in a text

In [20]:

```
fig,(ax1,ax2)=plt.subplots(1,2,figsize=(20,10))
word=df[df['Label']==1]['text'].str.split().apply(lambda x : [len(i) for i in x])
sns.distplot(word.map(lambda x: np.mean(x)),ax=ax1,color='green')
ax1.set_title('Real text')
word=df[df['Label']==0]['text'].str.split().apply(lambda x : [len(i) for i in x])
sns.distplot(word.map(lambda x: np.mean(x)),ax=ax2,color='red')
ax2.set_title('Fake text')
fig.suptitle('Average word length in each text')
```

/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/seaborn/distributions.py:2557: FutureWarning: `distplot` is a deprecated function and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

```
warnings.warn(msg, FutureWarning)
```

/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/numpy/core/fromnumeric.py:3372: RuntimeWarning: Mean of empty slice.

```
return _methods._mean(a, axis=axis, dtype=dtype,
```

/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/numpy/core/\_methods.py:170: RuntimeWarning: invalid value encountered in double\_scalars

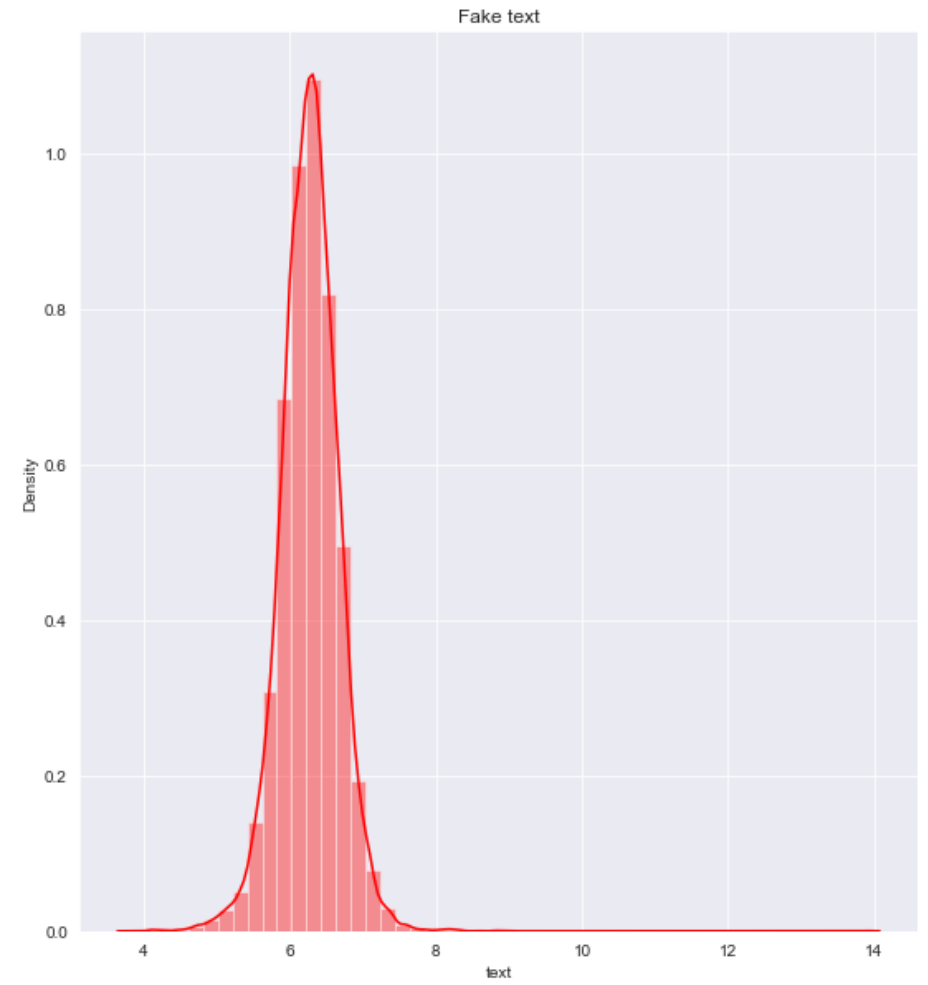
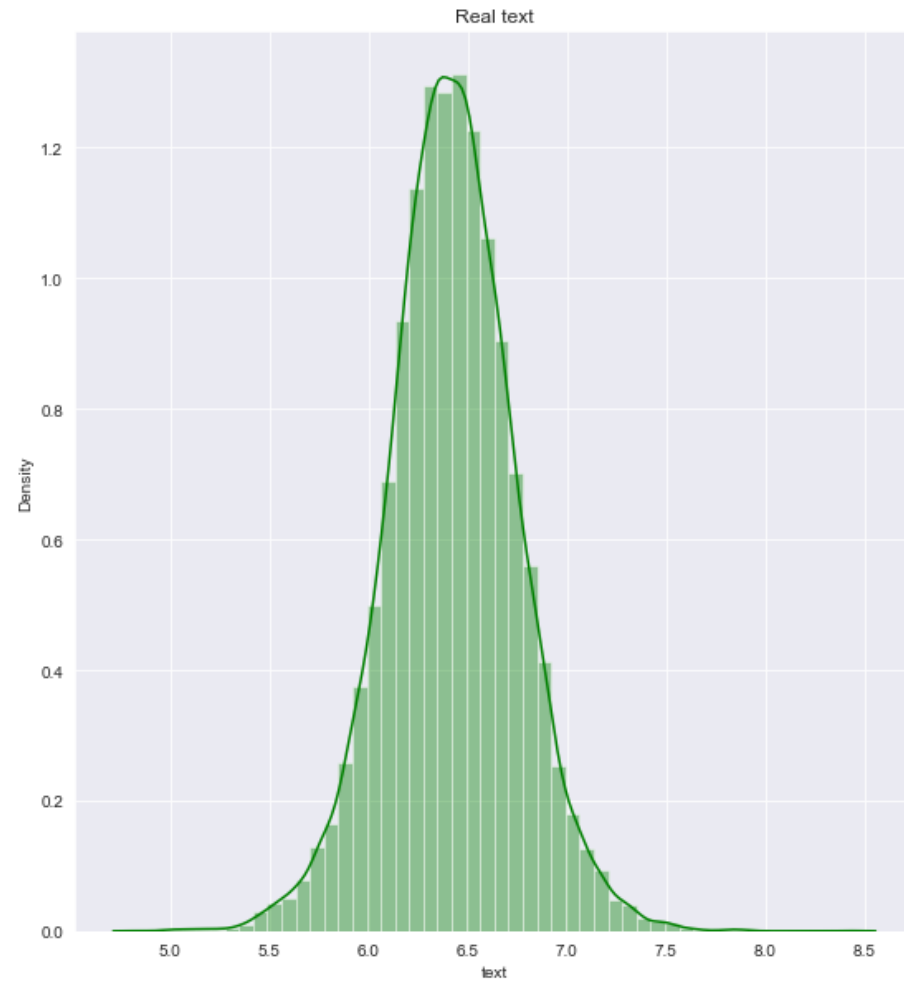
```
ret = ret.dtype.type(ret / rcount)
```

/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/seaborn/distributions.py:2557: FutureWarning: `distplot` is a deprecated function and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

```
warnings.warn(msg, FutureWarning)
```

Out[20]: Text(0.5, 0.98, 'Average word length in each text')

Average word length in each text



```
In [21]: def get_corpus(text):
          words = []
          for i in text:
              for j in i.split():
                  words.append(j.strip())
          return words
          corpus = get_corpus(df.text)
          corpus[:5]
```

```
Out[21]: ['washington', 'head', 'conservative', 'republican', 'faction']
```

```
In [22]: from collections import Counter
          counter = Counter(corpus)
          most_common = counter.most_common(10)
          most_common = dict(most_common)
          most_common
```

```
Out[22]: {'said': 130211,
          'trump': 128205,
          'u': 68141,
          'state': 57846,
          'would': 55292,
          'president': 53252,
          'people': 41187,
          'republican': 39560,
          'one': 36974,
          'year': 33220}
```

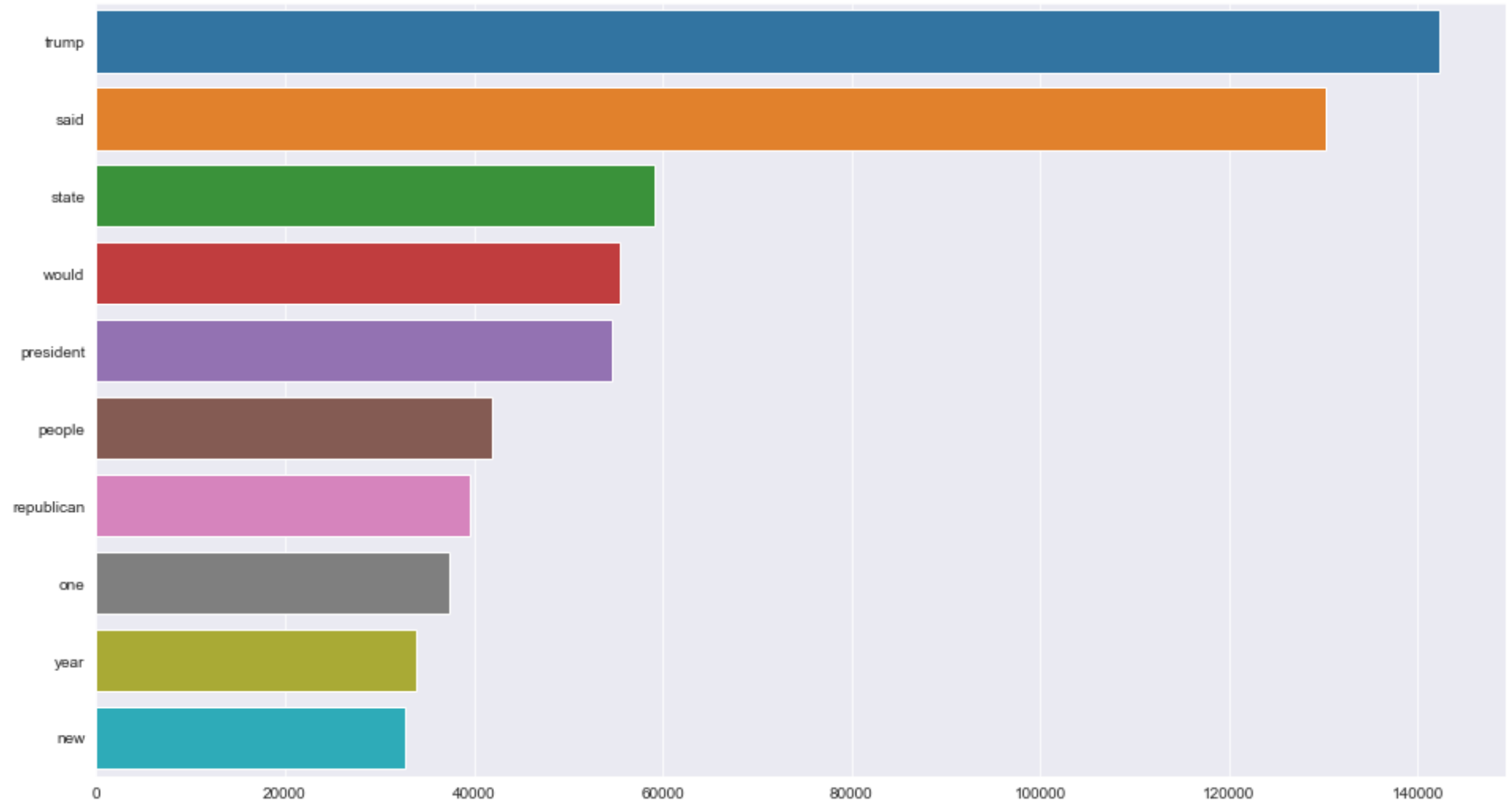
```
In [23]: from sklearn.feature_extraction.text import CountVectorizer
          def get_top_text_ngrams(corpus, n, g):
              vec = CountVectorizer(ngram_range=(g, g)).fit(corpus)
              bag_of_words = vec.transform(corpus)
              sum_words = bag_of_words.sum(axis=0)
              words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.items()]
              words_freq = sorted(words_freq, key = lambda x: x[1], reverse=True)
              return words_freq[:n]
```



# Unigram Analysis

```
In [24]: plt.figure(figsize = (16,9))
most_common_uni = get_top_text_ngrams(df.text,10,1)
most_common_uni = dict(most_common_uni)
sns.barplot(x=list(most_common_uni.values()),y=list(most_common_uni.keys()))
```

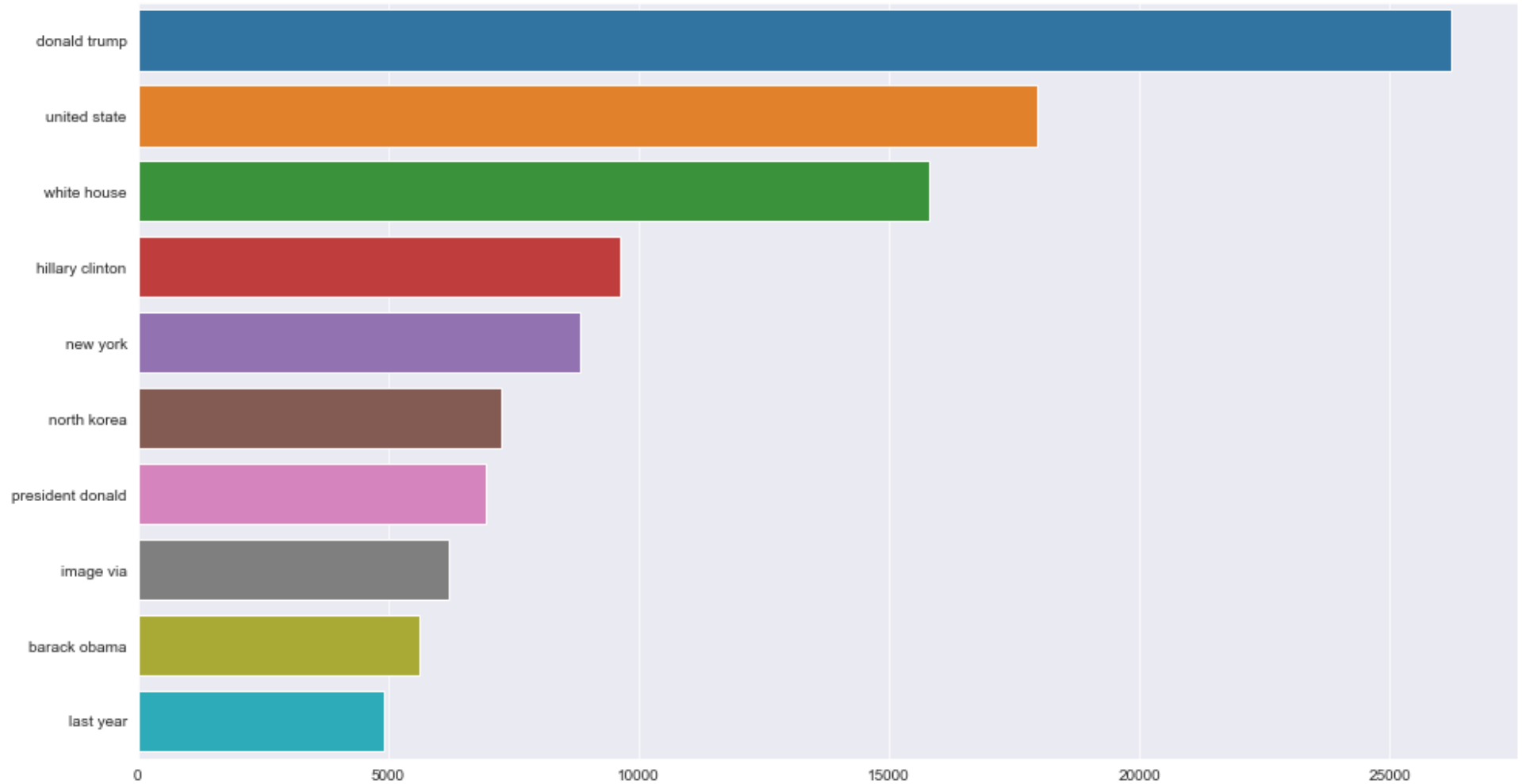
Out[24]: <AxesSubplot:>



## Bigram Analysis

```
In [25]: plt.figure(figsize = (16,9))
most_common_bi = get_top_text_ngrams(df.text,10,2)
most_common_bi = dict(most_common_bi)
sns.barplot(x=list(most_common_bi.values()),y=list(most_common_bi.keys()))
```

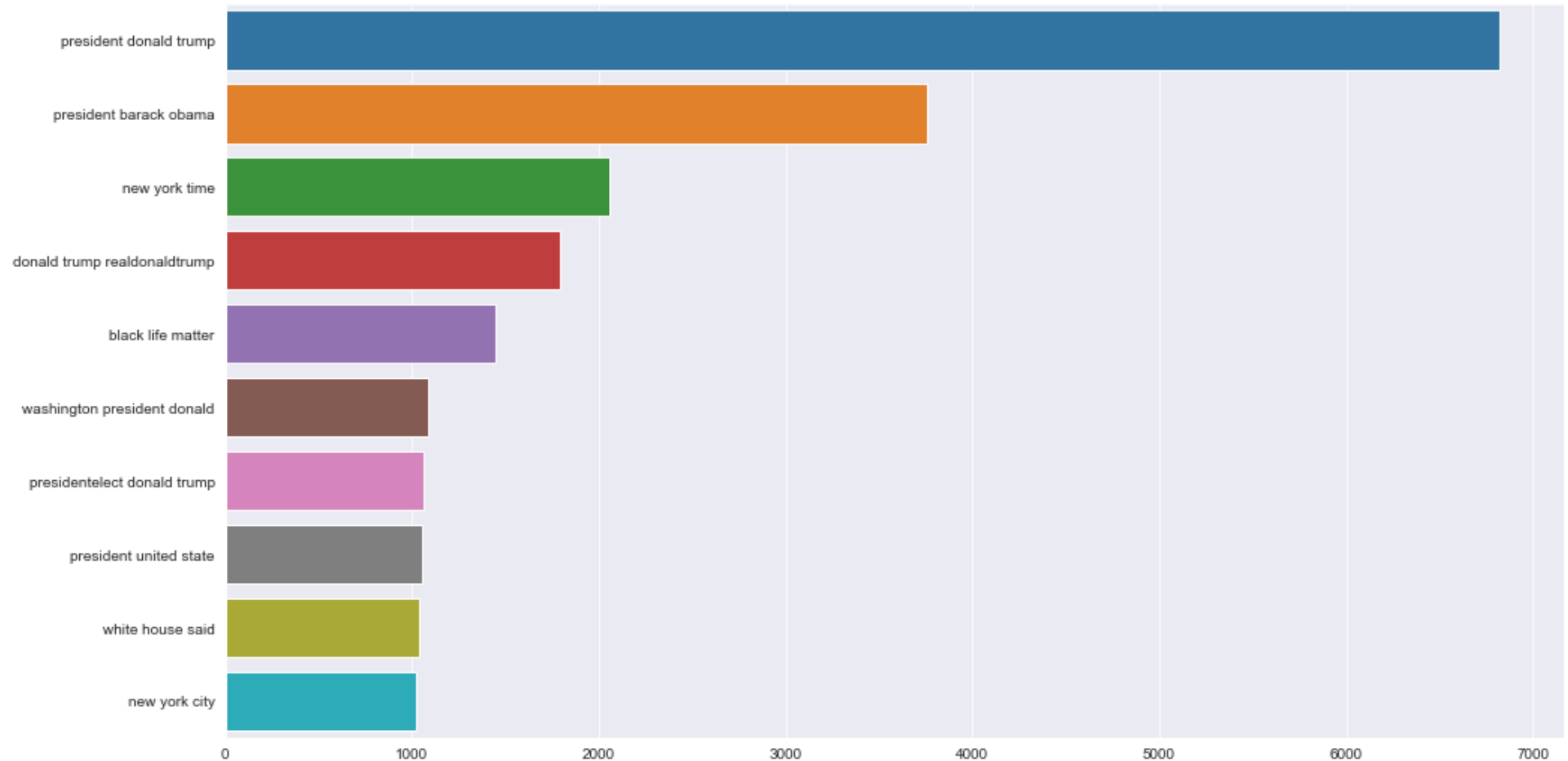
Out[25]: <AxesSubplot:>



# Trigram Analysis

```
In [26]: plt.figure(figsize = (16,9))
most_common_tri = get_top_text_ngrams(df.text,10,3)
most_common_tri = dict(most_common_tri)
sns.barplot(x=list(most_common_tri.values()),y=list(most_common_tri.keys()))
```

Out[26]: <AxesSubplot:>



## Splitting the data into 2 parts - training and testing data

```
In [27]: x_train,x_test,y_train,y_test = train_test_split(df.text,df.Label,random_state = 0)
```

```
In [28]: max_features = 10000
maxlen = 300
```

## Tokenizing Text

```
In [29]: tokenizer = text.Tokenizer(num_words=max_features)
tokenizer.fit_on_texts(x_train)
tokenized_train = tokenizer.texts_to_sequences(x_train)
x_train = sequence.pad_sequences(tokenized_train, maxlen=maxlen)
```

```
In [30]: tokenized_test = tokenizer.texts_to_sequences(x_test)
X_test = sequence.pad_sequences(tokenized_test, maxlen=maxlen)
```

## USING GloVe EMBEDDINGS FOR THE MODEL

```
In [31]: EMBEDDING_FILE = '/Users/moe/Desktop/glove/glove.6B.100d.txt'
```

```
In [32]: def get_coefs(word, *arr):
    return word, np.asarray(arr, dtype='float32')
embeddings_index = dict(get_coefs(*o.rstrip().rsplit(' ')) for o in open(EMBEDDING_FILE))
```

```
In [33]: all_embs = np.stack(embeddings_index.values())
emb_mean, emb_std = all_embs.mean(), all_embs.std()
embed_size = all_embs.shape[1]

word_index = tokenizer.word_index
nb_words = min(max_features, len(word_index))
embedding_matrix = embedding_matrix = np.random.normal(emb_mean, emb_std, (nb_words, embed_size))
for word, i in word_index.items():
    if i >= max_features: continue
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None: embedding_matrix[i] = embedding_vector
```

/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/IPython/core/interactiveshell.py:3361: FutureWarning: arrays to stack must be passed as a "sequence" type such as list or tuple. Support for non-sequence iterables such as generators is deprecated as of NumPy 1.16 and will raise an error in the future.

```
if (await self.run_code(code, result, async_=asy)):
```

## Model Parameters

```
In [34]: batch_size = 256
epochs = 10
embed_size = 100
```

```
In [35]: learning_rate_reduction = ReduceLROnPlateau(monitor='val_accuracy', patience = 2, verbose=1, factor=0.5, min_lr=0.0
```

# TRAINING THE MODEL

```
In [36]: model = Sequential()
#Non-trainable embeddidng layer
model.add(Embedding(max_features, output_dim=embed_size, weights=[embedding_matrix], input_length=maxlen, trainable=False))
#LSTM
model.add(LSTM(units=128, return_sequences = True, recurrent_dropout = 0.25, dropout = 0.25))
model.add(LSTM(units=64, recurrent_dropout = 0.1, dropout = 0.1))
model.add(Dense(units = 32, activation = 'relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer=keras.optimizers.Adam(learning_rate = 0.01), loss='binary_crossentropy', metrics=['accuracy'])
```

```
In [37]: model.summary()
```

Model: "sequential"

| Layer (type)                    | Output Shape     | Param # |
|---------------------------------|------------------|---------|
| =====                           |                  |         |
| embedding (Embedding)           | (None, 300, 100) | 1000000 |
| -----                           |                  |         |
| lstm (LSTM)                     | (None, 300, 128) | 117248  |
| -----                           |                  |         |
| lstm_1 (LSTM)                   | (None, 64)       | 49408   |
| -----                           |                  |         |
| dense (Dense)                   | (None, 32)       | 2080    |
| -----                           |                  |         |
| dense_1 (Dense)                 | (None, 1)        | 33      |
| =====                           |                  |         |
| Total params: 1,168,769         |                  |         |
| Trainable params: 168,769       |                  |         |
| Non-trainable params: 1,000,000 |                  |         |

```
In [38]: history = model.fit(x_train, y_train, batch_size = batch_size, validation_data = (x_test,y_test), epochs = epochs)
```

```

Epoch 1/10
132/132 [=====] - 551s 4s/step - loss: 0.2029 - accuracy: 0.9149 - val_loss: 0.0744 - val
_accuracy: 0.9718
Epoch 2/10
132/132 [=====] - 503s 4s/step - loss: 0.0588 - accuracy: 0.9789 - val_loss: 0.0393 - val
_accuracy: 0.9864
Epoch 3/10
132/132 [=====] - 527s 4s/step - loss: 0.0376 - accuracy: 0.9877 - val_loss: 0.0312 - val
_accuracy: 0.9904
Epoch 4/10
132/132 [=====] - 767s 6s/step - loss: 0.0243 - accuracy: 0.9920 - val_loss: 0.0250 - val
_accuracy: 0.9908
Epoch 5/10
132/132 [=====] - 801s 6s/step - loss: 0.0165 - accuracy: 0.9943 - val_loss: 0.0168 - val
_accuracy: 0.9946
Epoch 6/10
132/132 [=====] - 463s 4s/step - loss: 0.0130 - accuracy: 0.9955 - val_loss: 0.0163 - val
_accuracy: 0.9953
Epoch 7/10
132/132 [=====] - 446s 3s/step - loss: 0.0150 - accuracy: 0.9949 - val_loss: 0.0195 - val
_accuracy: 0.9947
Epoch 8/10
132/132 [=====] - 458s 3s/step - loss: 0.0129 - accuracy: 0.9959 - val_loss: 0.0188 - val
_accuracy: 0.9945

Epoch 00008: ReduceLROnPlateau reducing learning rate to 0.004999999888241291.
Epoch 9/10
132/132 [=====] - 445s 3s/step - loss: 0.0068 - accuracy: 0.9979 - val_loss: 0.0109 - val
_accuracy: 0.9970
Epoch 10/10
132/132 [=====] - 446s 3s/step - loss: 0.0045 - accuracy: 0.9986 - val_loss: 0.0102 - val
_accuracy: 0.9972

```

## ANALYSIS AFTER TRAINING OF MODEL

In [39]:

```

print("Accuracy of the model on Training Data is - " , model.evaluate(x_train,y_train)[1]*100 , "%")
print("Accuracy of the model on Testing Data is - " , model.evaluate(X_test,y_test)[1]*100 , "%")

```

```
1053/1053 [=====] - 113s 108ms/step - loss: 6.6417e-04 - accuracy: 0.9998
Accuracy of the model on Training Data is - 99.98218417167664 %
351/351 [=====] - 38s 108ms/step - loss: 0.0102 - accuracy: 0.9972
Accuracy of the model on Testing Data is - 99.7238278388977 %
```

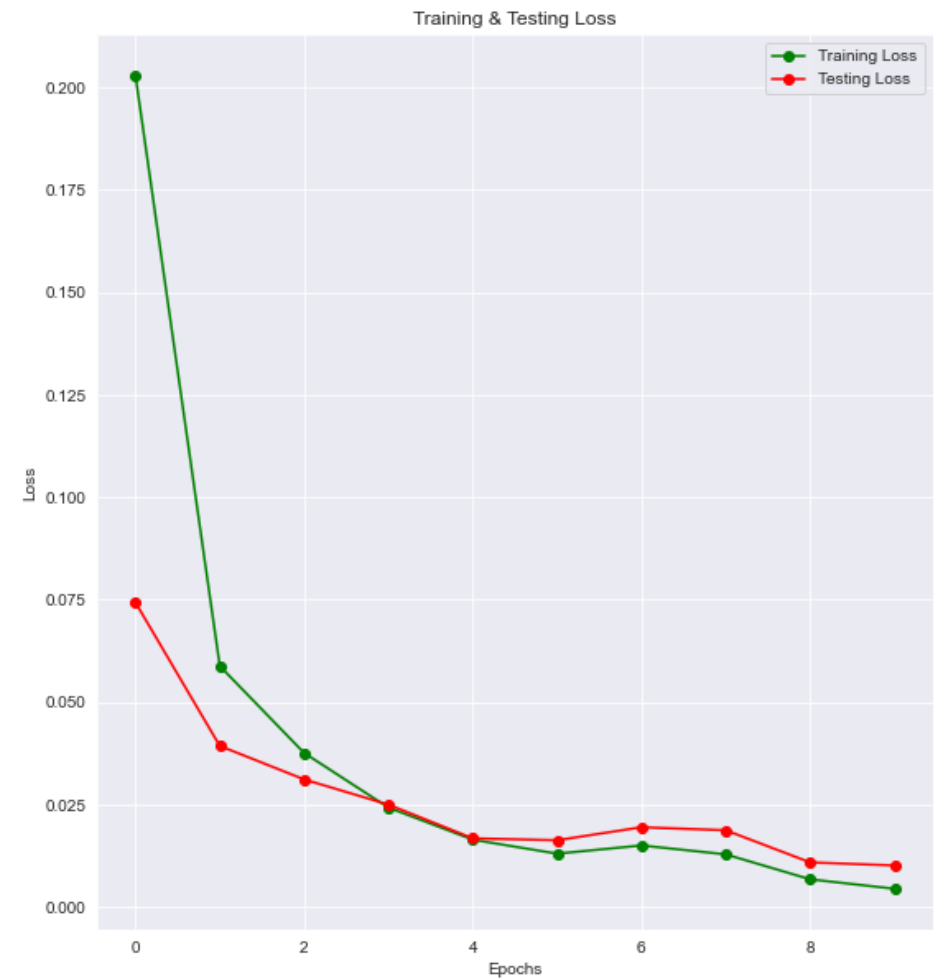
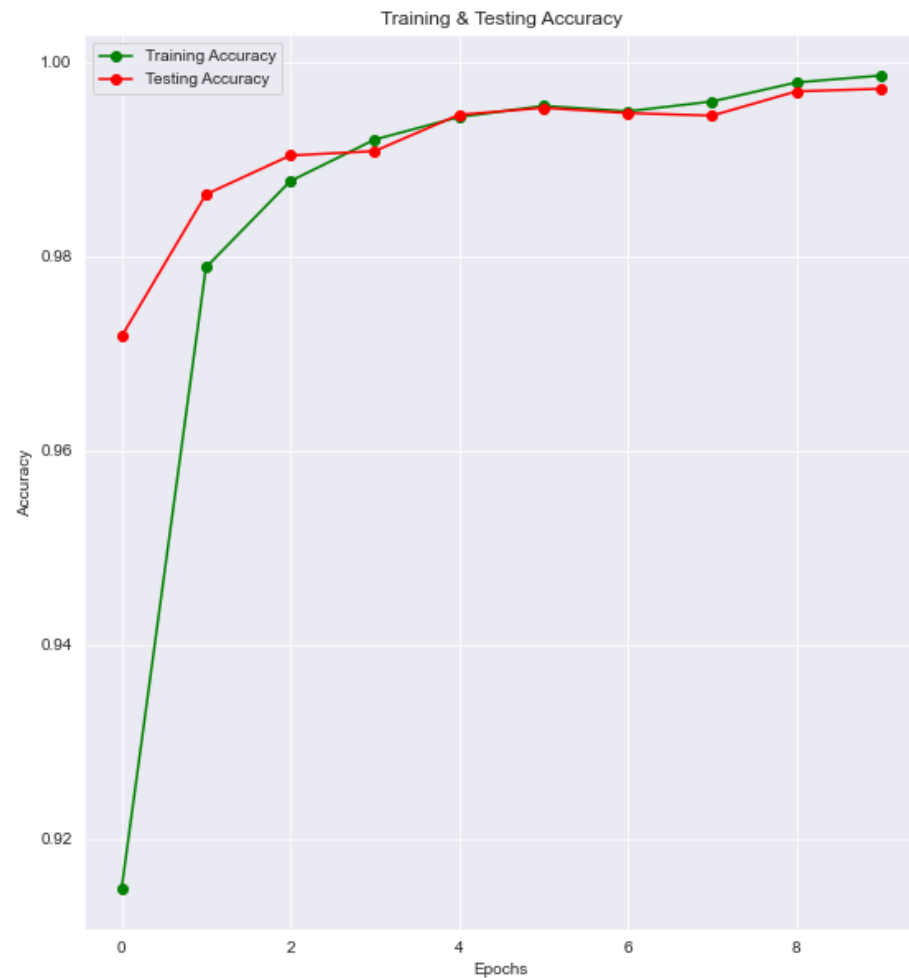
In [40]:

```
epochs = [i for i in range(10)]
fig, ax = plt.subplots(1,2)
train_acc = history.history['accuracy']
train_loss = history.history['loss']
val_acc = history.history['val_accuracy']
val_loss = history.history['val_loss']
fig.set_size_inches(20,10)

ax[0].plot(epochs, train_acc, 'go-', label = 'Training Accuracy')
ax[0].plot(epochs, val_acc, 'ro-', label = 'Testing Accuracy')
ax[0].set_title('Training & Testing Accuracy')
ax[0].legend()
ax[0].set_xlabel("Epochs")
ax[0].set_ylabel("Accuracy")

ax[1].plot(epochs, train_loss, 'go-', label = 'Training Loss')
ax[1].plot(epochs, val_loss, 'ro-', label = 'Testing Loss')
ax[1].set_title('Training & Testing Loss')
ax[1].legend()
ax[1].set_xlabel("Epochs")
ax[1].set_ylabel("Loss")
plt.show()
```





```
In [41]: pred = model.predict_classes(X_test)
```

/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/tensorflow/python/keras/engine/sequential.py:455: UserWarning: `model.predict\_classes()` is deprecated and will be removed after 2021-01-01. Please use instead: `np.argmax(model.predict(x), axis=-1)`, if your model does multi-class classification (e.g. if it uses a `softmax` last-layer activation). \* `(model.predict(x) > 0.5).astype("int32")`, if your model does binary classification (e.g. if it uses a `sigmoid` last-layer activation).

warnings.warn("`model.predict\_classes()` is deprecated and ")

```
In [42]: print(classification_report(y_test, pred, target_names = ['Fake', 'Real']))
```

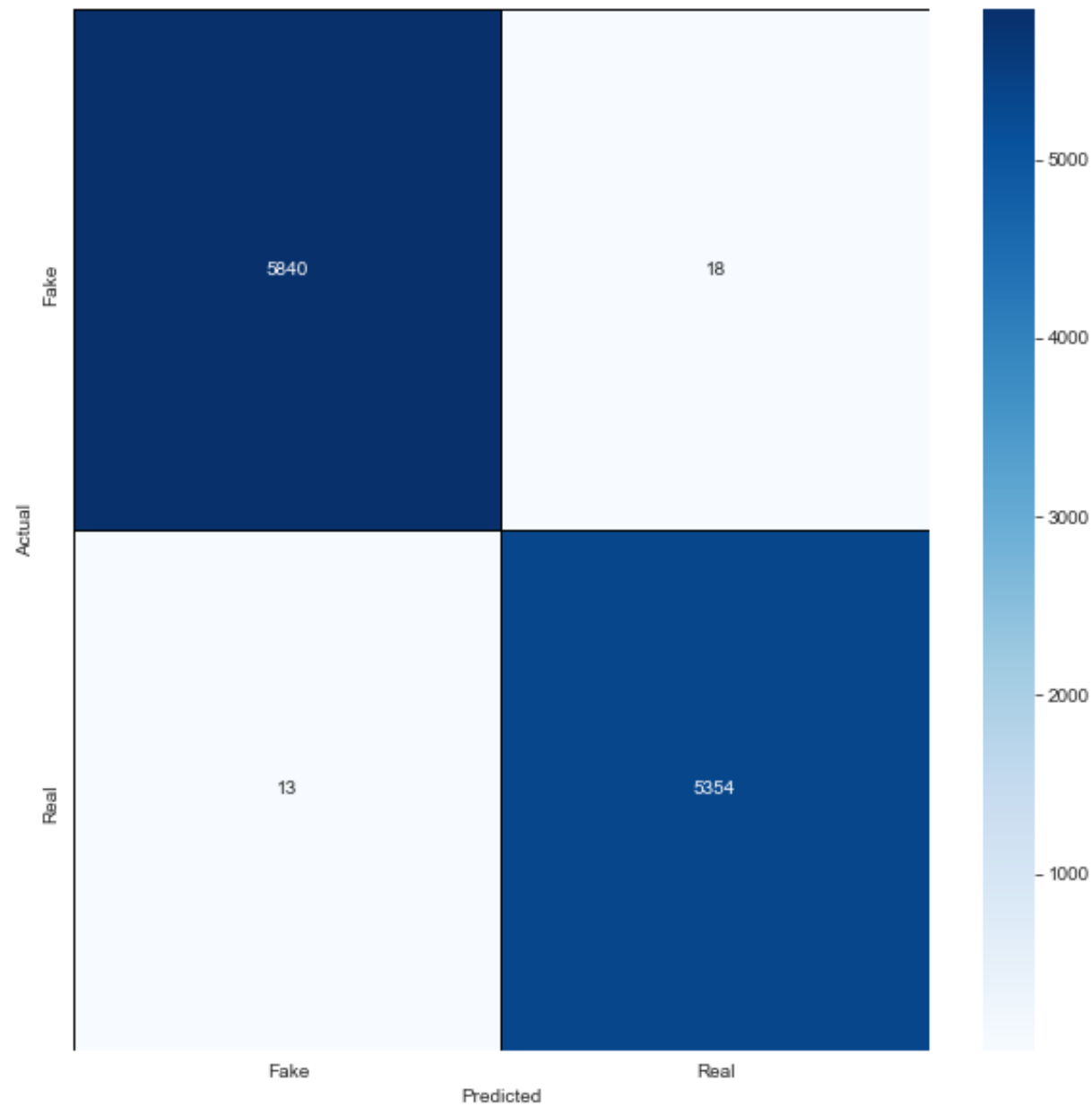
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Fake         | 1.00      | 1.00   | 1.00     | 5858    |
| Real         | 1.00      | 1.00   | 1.00     | 5367    |
| accuracy     |           |        | 1.00     | 11225   |
| macro avg    | 1.00      | 1.00   | 1.00     | 11225   |
| weighted avg | 1.00      | 1.00   | 1.00     | 11225   |

```
In [43]: cm = confusion_matrix(y_test, pred)
```

```
In [44]: cm = pd.DataFrame(cm , index = ['Fake', 'Real'] , columns = ['Fake', 'Real'])
```

```
In [45]: plt.figure(figsize = (10,10))
sns.heatmap(cm, cmap= "Blues", linecolor = 'black' , linewidth = 1 , annot = True, fmt='', xticklabels = ['Fake', 'Real'], yticklabels = ['Fake', 'Real'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
Out[45]: Text(70.0, 0.5, 'Actual')
```



saving weights of our model and label map, pickle our tokenizer.

```
In [46]: model.save_weights("trained_model.h5")
```

```
In [47]: import pickle
with open("tokenizer.pickle",mode="wb") as F:
    pickle.dump(tokenizer,F)
```

```
In [48]: import json
label_map = {0:"Fake",
             1:"Real"
            }

json.dump(label_map,open("label_map.json",mode="w"))
```

## creating a deployable model

In [49]:

```
class DeployModel():

    def __init__(self, weights_path, tokenizer_path, seq_length, label_map_path):

        self.model = model
        self.model.load_weights(weights_path)
        self.tokenizer = pickle.load(open(tokenizer_path, mode="rb"))
        self.seq_len = seq_length
        self.label_map = json.load(open(label_map_path))

    def _prepare_data(self, text):

        cleaned = denoise_text(text)
        tokenized = self.tokenizer.texts_to_sequences([cleaned])
        padded = pad_sequences(tokenized, maxlen=self.seq_len)
        return padded

    def _predict(self, text):

        text = self._prepare_data(text)
        pred = int(self.model.predict_classes(text)[0])
        return str(pred)

    def result(self, text):

        pred = self._predict(text)
        return self.label_map[pred]
```

In [50]:

```
deploy_model = DeployModel(weights_path="./trained_model.h5",
                             tokenizer_path="./tokenizer.pickle",
                             seq_length=maxlen,
                             label_map_path="./label_map.json"
                             )
```

## SCRAPED NEWS FOR 2020

```
In [51]: A=pd.read_csv('/Users/moe/Desktop/Scraped News/ALLNEWS.csv')
A['text']=(A['Headline'] + ' ' + A['Body'])
A=A.drop('Body',axis=1)

print (A.isnull().sum())
A = A.dropna()
print (A.isnull().sum())
```

```
Date          13095
Headline       25
Source         0
text          41
dtype: int64
Date          0
Headline      0
Source        0
text          0
dtype: int64
```

```
In [83]: A['text']=A['text'].apply(denoise_text)
```

## DATA VISUALIZATION

```
In [61]: A
```

Out[61]:

|       | Date               | Headline                                          | Source   | text                                              |
|-------|--------------------|---------------------------------------------------|----------|---------------------------------------------------|
| 0     | 2020-01-01         | Protesters attack American embassy in Iraq        | Dawn     | protesters attack american embassy iraq baghda... |
| 1     | 2020-01-01         | Biggest urbanisation in history under way, say... | Dawn     | biggest urbanisation history way says study lo... |
| 2     | 2020-01-01         | UK govt raises minimum wage                       | Dawn     | uk govt raises minimum wage london britain's m... |
| 3     | 2020-01-01         | Plan to rename Heathrow after Churchill never ... | Dawn     | plan rename heathrow churchill never moved ahe... |
| 4     | 2020-01-01         | Raging wildfires trap 4,000 at Australian town... | Dawn     | raging wildfires trap australian town's waterf... |
| ...   | ...                | ...                                               | ...      | ...                                               |
| 68282 | 20th December 2020 | Chelsea footballer Billy Gilmour donates signe... | Scotsman | chelsea footballer billy gilmour donates signe... |
| 68283 | 20th December 2020 | 'Brown noising up two average Rangers fans' - ... | Scotsman | brown noising two average rangers fans celtic ... |
| 68284 | 20th December 2020 | Police Scotland to double its presence in bord... | Scotsman | police scotland double presence border areas c... |
| 68285 | 20th December 2020 | Edinburgh Festival Fringe commits to become tr... | Scotsman | edinburgh festival fringe commits become trail... |
| 68286 | 20th December 2020 | Warm tribute paid to an island's legendary las... | Scotsman | warm tribute paid islands legendary last black... |

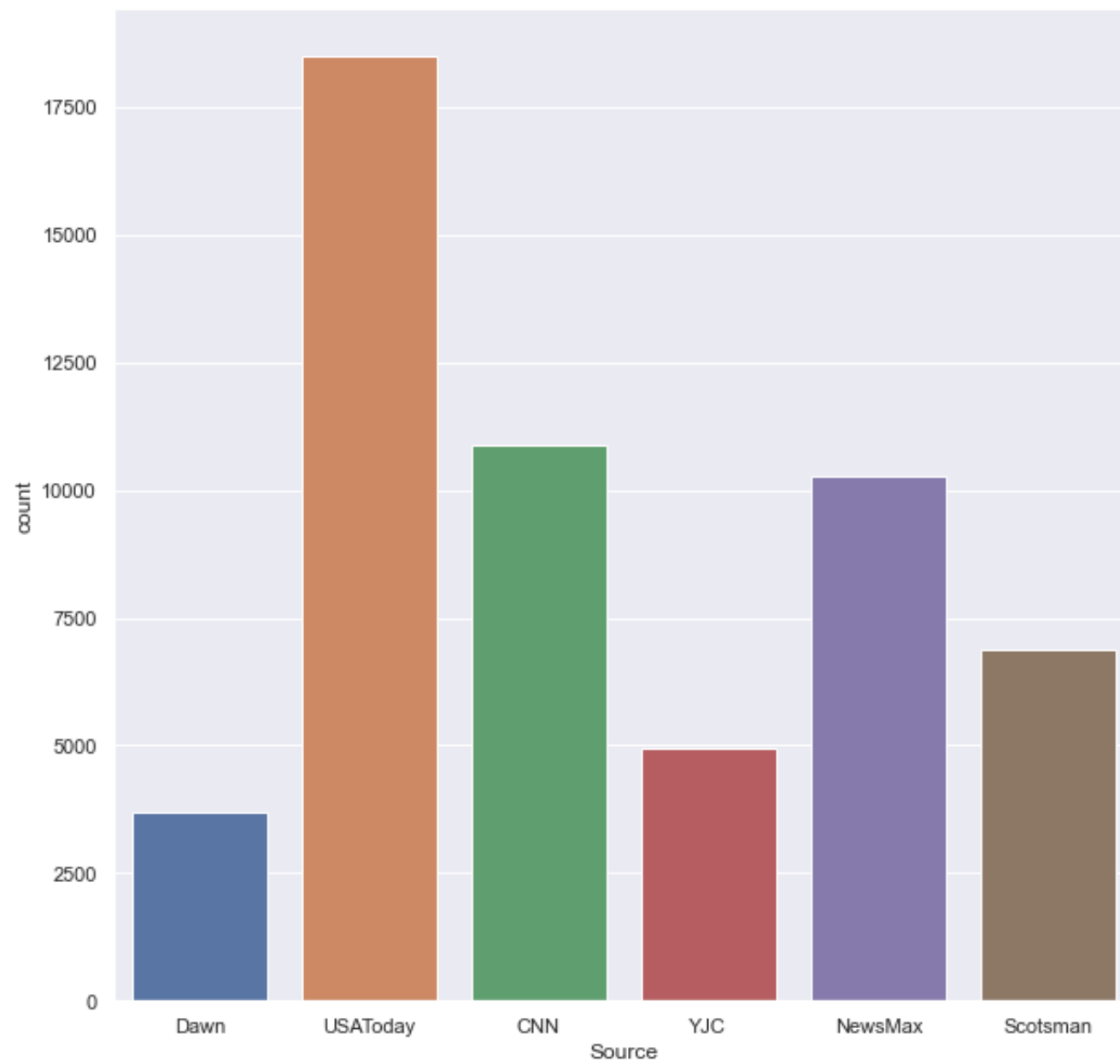
55178 rows x 4 columns

In [63]:

```
sns.set_style("darkgrid")
sns.set(rc = {'figure.figsize':(10,10)})
sns.countplot(A.Source)
```

```
/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/seaborn/_decorators.py:36: FutureWarning
: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `
data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

Out[63]: <AxesSubplot:xlabel='Source', ylabel='count'>







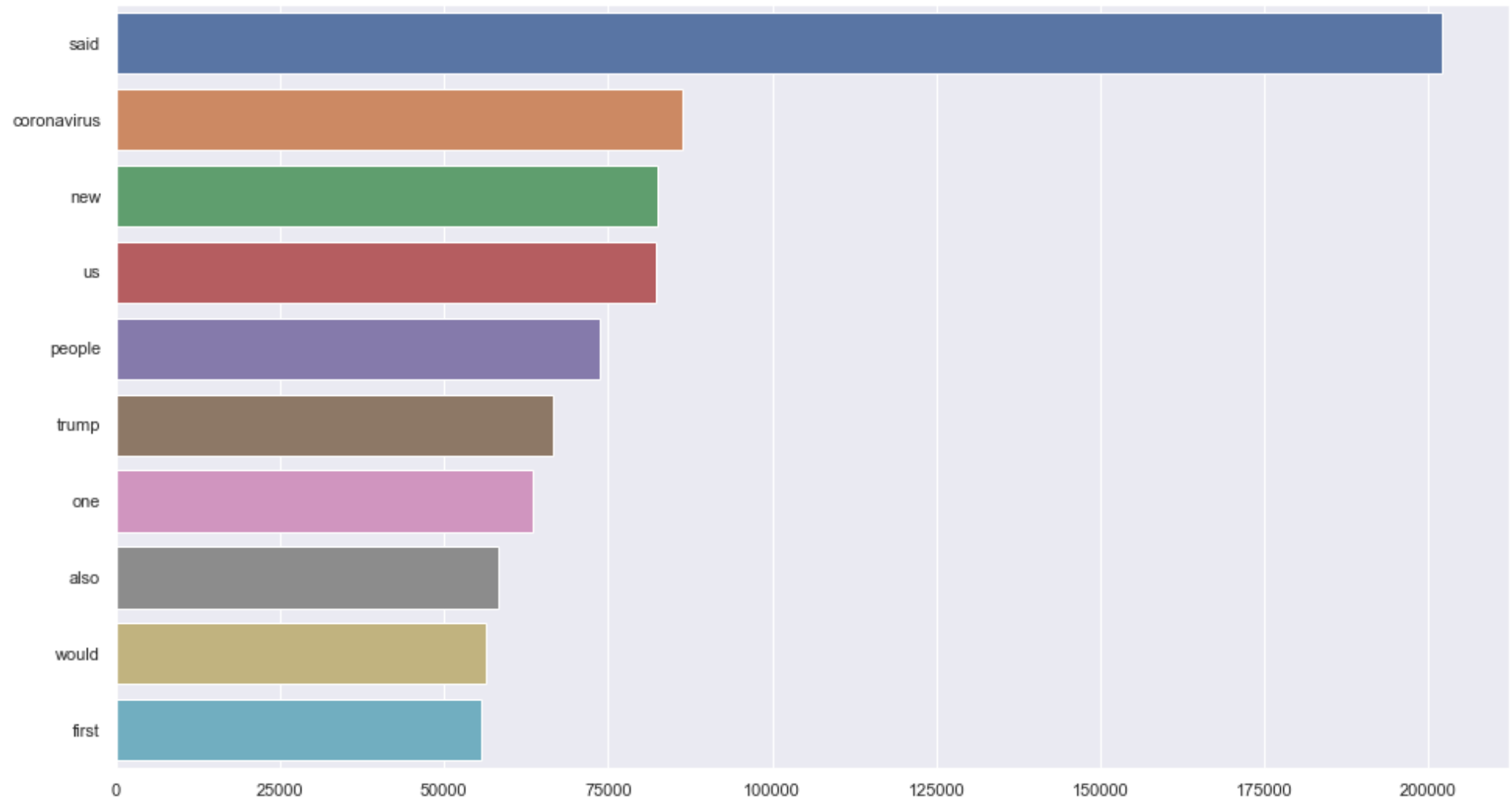
```
Out[78]: ['protesters', 'attack', 'american', 'embassy', 'iraq']
```

```
In [79]: counter = Counter(corpus)
most_common = counter.most_common(10)
most_common = dict(most_common)
most_common
```

```
Out[79]: {'said': 201064,
'coronavirus': 85808,
'new': 82229,
'us': 80993,
'people': 71471,
'one': 62368,
'trump': 62204,
'also': 58142,
'would': 56186,
'first': 55448}
```

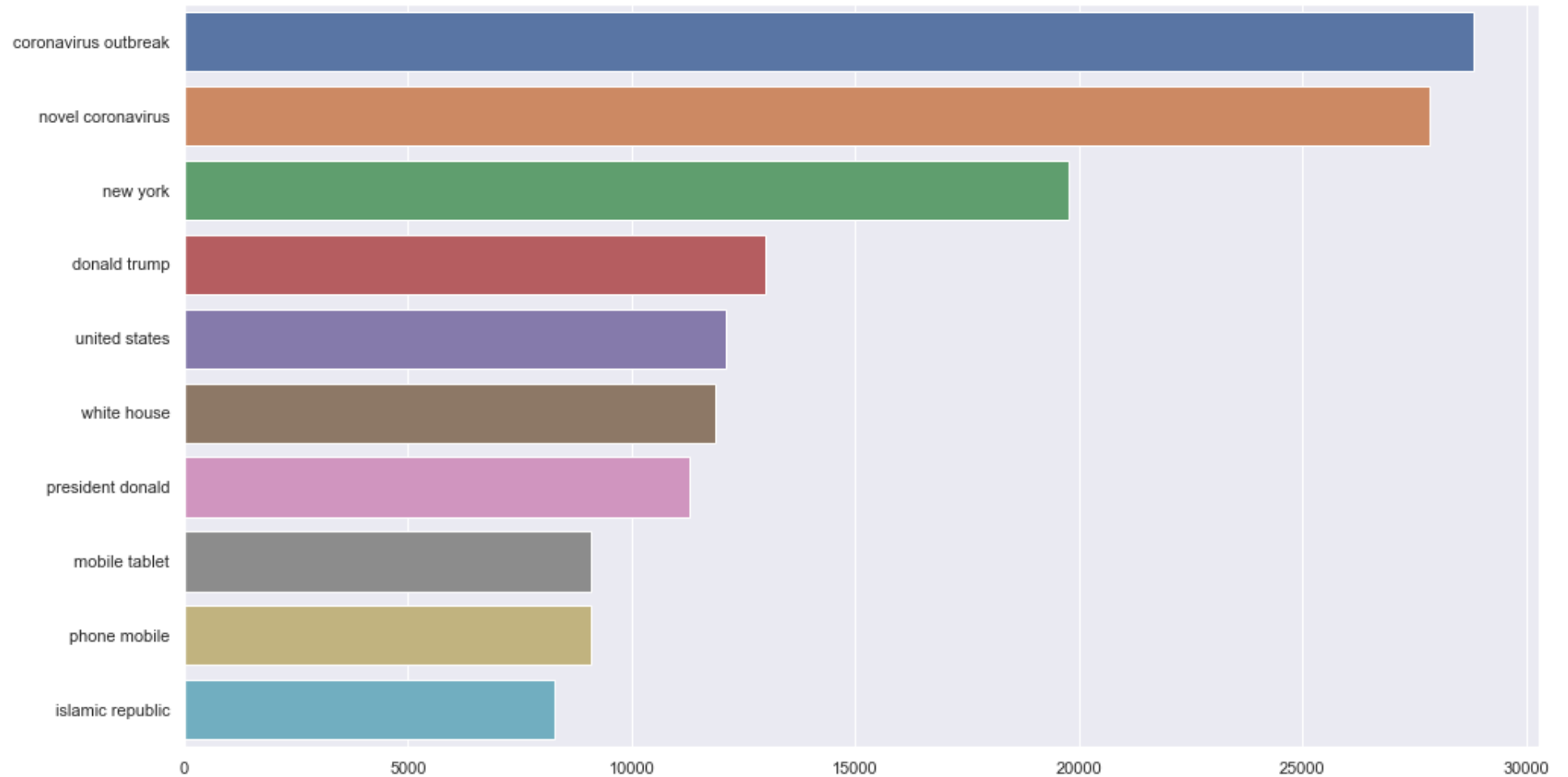
```
In [72]: plt.figure(figsize = (16,9))
most_common_uni = get_top_text_ngrams(A.text,10,1)
most_common_uni = dict(most_common_uni)
sns.barplot(x=list(most_common_uni.values()),y=list(most_common_uni.keys()))
fig.suptitle('Unigram analysis of 2020 news articles')
```

Out[72]: Text(0.5, 0.98, 'Unigram analysis of 2020 news articles')



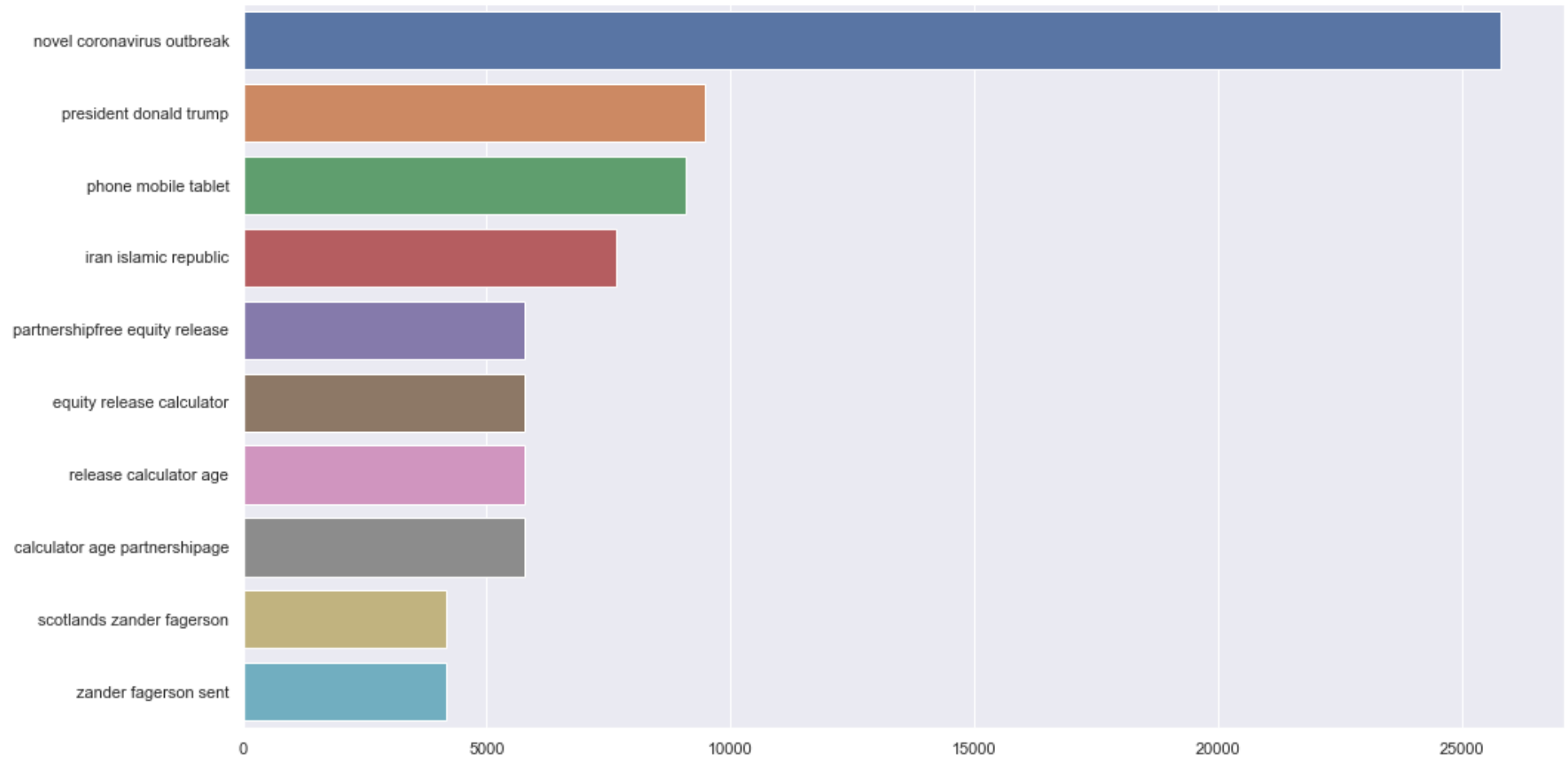
```
In [80]: plt.figure(figsize = (16,9))
most_common_bi = get_top_text_ngrams(A.text,10,2)
most_common_bi = dict(most_common_bi)
sns.barplot(x=list(most_common_bi.values()),y=list(most_common_bi.keys()))
fig.suptitle('Bnigram analysis of 2020 news articles')
```

Out[80]: Text(0.5, 0.98, 'Bnigram analysis of 2020 news articles')



```
In [84]: plt.figure(figsize = (16,9))
most_common_tri = get_top_text_ngrams(A.text,10,3)
most_common_tri = dict(most_common_tri)
sns.barplot(x=list(most_common_tri.values()),y=list(most_common_tri.keys()))
fig.suptitle('Trigram analysis of 2020 news articles')
```

Out[84]: Text(0.5, 0.98, 'Trigram analysis of 2020 news articles')



## applying the model to our news

```
In [92]: from sklearn.utils import shuffle
A = shuffle(A)
A.reset_index(inplace=True, drop=True)
```

In [94]:

```
for i in range(len(A)):
    news=A.at[i,'text']
    r = deploy_model.result(news)
    if r == 'Real':
        A.at[i,'Result'] = 'real'
    if r == 'Fake':
        A.at[i,'Result'] = 'fake'
```

```
/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/tensorflow/python/keras/engine/sequential.py:455: UserWarning: `model.predict_classes()` is deprecated and will be removed after 2021-01-01. Please use instead: * `np.argmax(model.predict(x), axis=-1)`, if your model does multi-class classification (e.g. if it uses a `softmax` last-layer activation). * `(model.predict(x) > 0.5).astype("int32")`, if your model does binary classification (e.g. if it uses a `sigmoid` last-layer activation).
  warnings.warn("`model.predict_classes()` is deprecated and '
```

In [95]:

A

| Out[95]: |       | Date             | Headline                                           | Source   |  | text                                              | Result |
|----------|-------|------------------|----------------------------------------------------|----------|--|---------------------------------------------------|--------|
|          | 0     | 2020-04-16       | Top 2020 prospect Jalen Green will head to NBA...  | USAToday |  | top prospect jalen green head nba g league ins... | fake   |
|          | 1     | 2020-02-15       | American woman, 83, tests positive for coronav...  | USAToday |  | american woman tests positive coronavirus dise... | real   |
|          | 2     | 2020-04-28       | Melissa McCarthy's dream about Mark Wahlberg m...  | CNN      |  | melissa mccarthys dream mark wahlberg may come... | fake   |
|          | 3     | 2020-01-17       | Biden Could Gain From Warren-Sanders Dispute       | NewsMax  |  | biden could gain warrensanders dispute former ... | fake   |
|          | 4     | 2020-08-23       | Larry King Mourns Loss of Two of His Adult Chi...  | NewsMax  |  | larry king mourns loss two adult children with... | fake   |
|          | ...   | ...              | ...                                                | ...      |  | ...                                               | ...    |
|          | 55173 | 2020-11-04       | Fox's usual 'Big Noon Kickoff' team won't be o...  | USAToday |  | foxs usual big noon kickoff team saturdays bro... | fake   |
|          | 55174 | 2020-05-06       | Sources: US Investigating Ex-Green Beret for V...  | NewsMax  |  | sources us investigating exgreen beret venezue... | real   |
|          | 55175 | 2020-11-25       | O'Brien Warns Biden 'Appeasement' of Iran Risk...  | YJC      |  | o'brien warns biden 'appeasement' iran risks m... | real   |
|          | 55176 | 2020-12-09       | 'Mars, here we come!!!': SpaceX's Starship prot... | USAToday |  | mars come spacexs starship prototype performs ... | fake   |
|          | 55177 | 25th August 2020 | Pat McCluskey: Kenny Dalglish leads tributes t...  | Scotsman |  | pat mccluskey kenny dalglish leads tributes fo... | fake   |

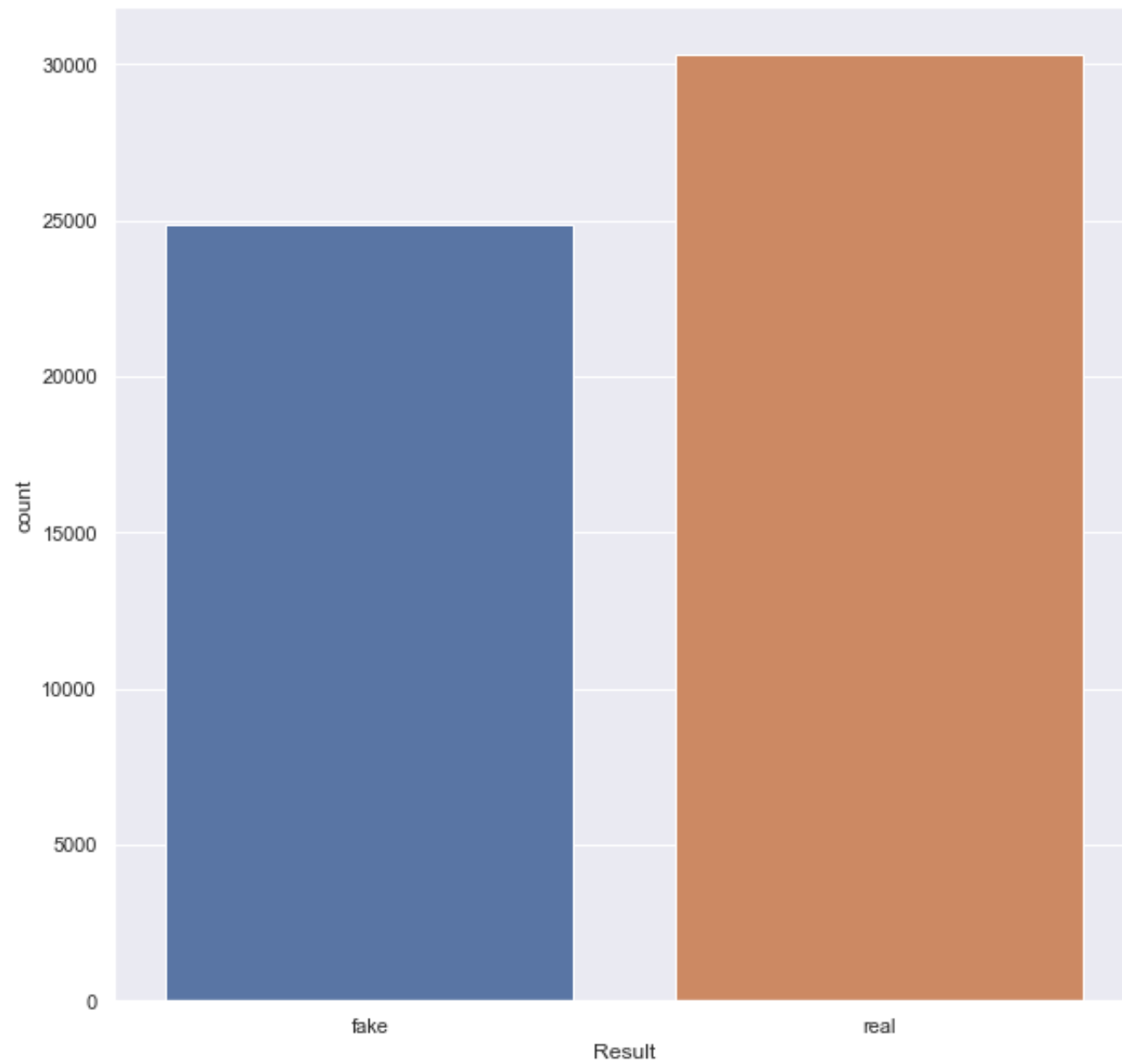
55178 rows x 5 columns

## ploting fake and real news in our dataset

```
In [96]: sns.set_style("darkgrid")
sns.set(rc = {'figure.figsize':(10,10)})
sns.countplot(A.Result)
```

```
/Users/moe/.pyenv/versions/3.8.10/envs/myvenv/lib/python3.8/site-packages/seaborn/_decorators.py:36: FutureWarning
: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `
data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.
warnings.warn(
```

```
Out[96]: <AxesSubplot:xlabel='Result', ylabel='count'>
```



ploting our labeled dataset



In [97]:

```
CNNreal=0
CNNfake=0
Dawnreal=0
Dawnfake=0
NewsMaxreal=0
NewsMaxfake=0
Scotsmanreal=0
Scotsmanfake=0
USATodayreal=0
USATodayfake=0
YJCrealm=0
YJCfake=0
```

In [98]:

```
for i in range(len(A)):
    if A.at[i, 'Result'] == 'real' and A.at[i, 'Source'] == 'CNN':
        CNNreal=CNNreal+1
    if A.at[i, 'Result'] == 'fake' and A.at[i, 'Source'] == 'CNN':
        CNNfake=CNNfake+1
    if A.at[i, 'Result'] == 'real' and A.at[i, 'Source'] == 'Dawn':
        Dawnreal=Dawnreal+1
    if A.at[i, 'Result'] == 'fake' and A.at[i, 'Source'] == 'Dawn':
        Dawnfake=Dawnfake+1
    if A.at[i, 'Result'] == 'real' and A.at[i, 'Source'] == 'NewsMax':
        NewsMaxreal=NewsMaxreal+1
    if A.at[i, 'Result'] == 'fake' and A.at[i, 'Source'] == 'NewsMax':
        NewsMaxfake=NewsMaxfake+1
    if A.at[i, 'Result'] == 'real' and A.at[i, 'Source'] == 'Scotsman':
        Scotsmanreal=Scotsmanreal+1
    if A.at[i, 'Result'] == 'fake' and A.at[i, 'Source'] == 'Scotsman':
        Scotsmanfake=Scotsmanfake+1
    if A.at[i, 'Result'] == 'real' and A.at[i, 'Source'] == 'USAToday':
        USATodayreal=USATodayreal+1
    if A.at[i, 'Result'] == 'fake' and A.at[i, 'Source'] == 'USAToday':
        USATodayfake=USATodayfake+1
    if A.at[i, 'Result'] == 'real' and A.at[i, 'Source'] == 'YJC':
        YJCrealm=YJCrealm+1
    if A.at[i, 'Result'] == 'fake' and A.at[i, 'Source'] == 'YJC':
        YJCfake=YJCfake+1
```

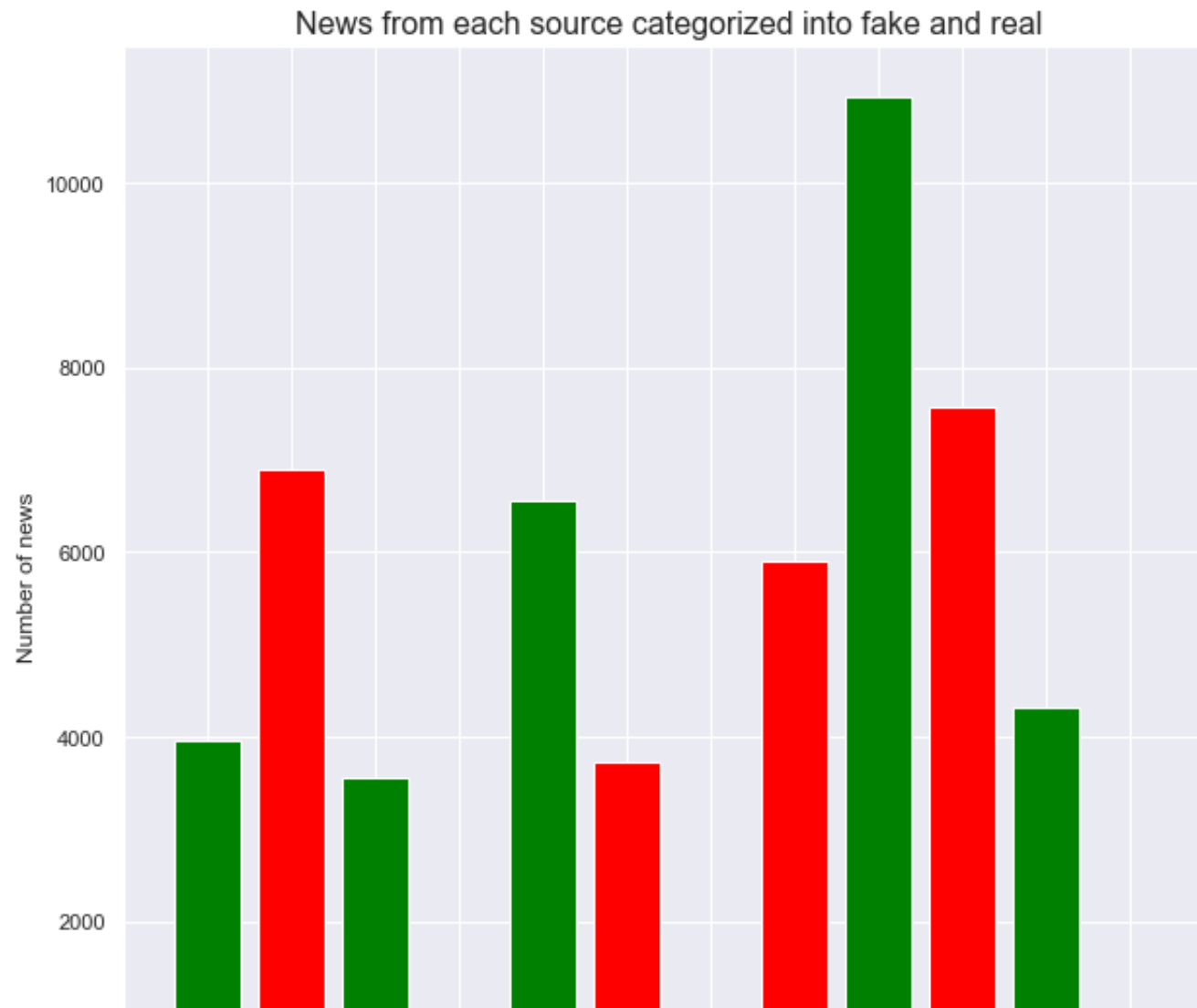
```
In [99]: A.Result.value_counts()
```

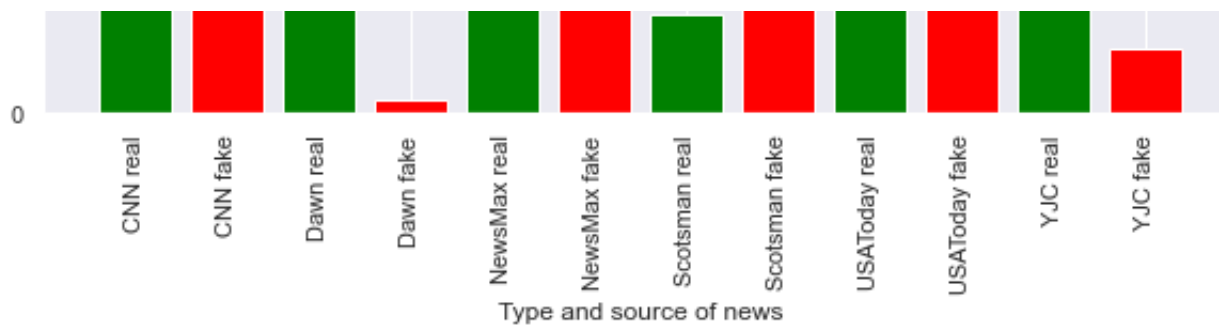
```
Out[99]: real      30331  
fake       24847  
Name: Result, dtype: int64
```

```
In [100... fp=[CNNreal,  
CNNfake,  
Dawnreal,  
Dawnfake,  
NewsMaxreal,  
NewsMaxfake,  
Scotsmanreal,  
Scotsmanfake,  
USATodayreal,  
USATodayfake,  
YJCrealm,  
YJCfake]  
fplabels=['CNN real',  
'CNN fake',  
'Dawn real',  
'Dawn fake',  
'NewsMax real',  
'NewsMax fake',  
'Scotsman real',  
'Scotsman fake',  
'USAToday real',  
'USAToday fake',  
'YJC real',  
'YJC fake']  
colors = ['green', 'red']
```

In [101...

```
plt.bar(fplabels,fp,color=colors)
plt.xticks(rotation = 90)
plt.title('News from each source categorized into fake and real',fontsize=16)
plt.xlabel('Type and source of news',fontsize=12)
plt.ylabel('Number of news',fontsize=12)
plt.grid(True)
plt.show()
```





## saving our labeled dataset

```
In [102... A.to_csv(r'/Users/moe/Desktop/Scraped News/ALLNEWSRESULT.csv' , index = False)
```

## saving fake and real datasets seperately

```
In [103... A=pd.read_csv('/Users/moe/Desktop/Scraped News/ALLNEWSRESULT.csv')
```

```
In [104... Areal, Afake = [v for _, v in A.groupby(A['Result'] != 'real')]
```

```
In [105... Areal.to_csv(r'/Users/moe/Desktop/Scraped News/ALLNEWSREAL.csv' , index = False)
Afake.to_csv(r'/Users/moe/Desktop/Scraped News/ALLNEWSFAKE.csv' , index = False)
```

```
In [ ]:
```